SCIENCES SORBONNE UNIVERSITÉ

UFR d'Ingénierie

23 septembre 2026

L2 Informatique (2026-2027)

Logique (UL2IN007)

TP

Thèmes 1 à 11

Thème 1.2.3 - TP

Objectifs

- Logique propositionnelle
  - syntaxe
  - tables de vérité
  - sémantique
  - Interprétations
  - Expressions booléennes
  - Satisfiabilité, validité
  - Conséquence sémantique, équivalence

Exercices

Le but de cette série d'exercices est d'écrire un programme Python qui interprète des formules logiques exprimées sous forme d'une chaîne de caractères.

Les formules sont construites inductivement selon les règles suivantes :
- `TRUE` et `FALSE` sont des formules logiques ;
- si F est une formule atomique, alors F est une formule logique ;
- si F est une formule logique, alors `NOT` F est une formule logique ;
- si F_1 et F_2 sont des formules logiques, alors F_1 `AND` F_2, F_1 `OR` F_2 et F_1 `IMPLIES` F_2 sont des formules logiques.

Les formules atomiques seront représentées par des chaînes de caractères. Les formules sont parenthésées pour éviter toute ambguïté. Voici quelques exemples de formules :
- `"bonjour"`
- `"TRUE IMPLIES FALSE"`
- `"(NOT b) AND (q OR r)"`

Exercice 1

Nous considérons dans un premier temps des formules construites uniquement à partir de `TRUE` et `FALSE`.

Lorsque l'on construit l'arbre de syntaxe abstraite d'une formule, on commence par identifier le connecteur principal, qui sera l'étiquette de la racine de l'arbre. Vous disposez d'une fonction

```
decompose(formule: str)-> (str, list[str])
```

qui prend une formule en paramètre et qui renvoie le couple formé du connecteur principal et de la liste des sous-formules (0, 1 ou 2). Par exemple :
- `decompose("p")` renvoie `('p', [])`
- `decompose("NOT b")` renvoie `('NOT', ['b'])`
- `decompose("(NOT b)AND (q OR r)")` renvoie `('AND', ['(NOT b)','(q OR r)'])`

Lorsque la formule est représentée par un AST,
- soit l'arbre est réduit à une feuille et la valeur de la formule correspond à l'évaluation de la feuille,
- soit la valeur est obtenue en évaluant le(s) sous-arbre(s) et en leur appliquant l'opération représentée à la racine. Les sous-arbres correspondent ici aux éléments de la liste renvoyée par `decompose`.

Les formules traitées dans cet exercice sont construites uniquement à partir de `TRUE` et `FALSE`.

Question 1
Écrivez une fonction récursive `eval_TF (formule : str)-> bool` qui prend en paramètre une formule et qui calcule la valeur booléenne obtenue lors de l'interprétation de cette formule. Par exemple,

- `eval_TF("TRUE AND FALSE")` renvoie `False`,
- `eval_TF("(TRUE AND FALSE)OR (TRUE OR (TRUE AND FALSE))")` renvoie `True`.

Si nécessaire, vous pouvez utiliser la fonction `strip` de la bibliothèque Python pour éliminer d'éventuels espaces inutiles en début ou fin de chaîne.

Exercice 2

Dans le cas général, une formule est construite à partir de formules atomiques. Pour interpréter la formule, il faut donc :

- identifier les formules atomiques qui y figurent,
- interpréter les formules atomiques puis interpréter la formule complète de manière récursive.

Pour identifier les formules atomiques, nous allons réutiliser la fonction

`decompose(formule: str)-> (str, list[str])`

définie dans l'exercice précédent. En effet, les formules atomiques (mais attention, `TRUE` et `FALSE` aussi) étiquettent les feuilles de l'arbre de syntaxe abstraite et celles-ci peuvent être facilement identifiées au moyen de la fonction :
`decompose("p")` renvoie `('p', [])`

Question 1
Écrivez la fonction `genere_fa(formule : str)-> set[str]` qui renvoie l'ensemble des formules atomiques contenues dans formule. Par exemple :

- `genere_fa("TRUE")` renvoie `set()`
- `genere_fa("p AND FALSE")` renvoie `{'p'}`
- `genere_fa("hello OR (aurevoir AND salut)")` renvoie `{'hello', 'aurevoir', 'salut'}`

Pour interpréter une formule, il faut commencer par interpréter les formules atomiques que nous venons d'isoler. Nous supposons que nous disposons d'un dictionnaire qui associe une valeur booléenne à chaque formule atomique.
Par exemple, `{'p': True, 'q': False, 'r': True, 'bonjour': True, 'a': False}` est une valuation qui peut être utilisée pour évaluer les formules `"p AND q"` ou `"r IMPLIES bonjour"`.

Question 2
Écrivez une fonction récursive `eval_formule(formule : str, valuation : dict)-> bool` qui prend en paramètre une formule et une valuation et qui renvoie la valeur booléenne correspondant à l'interprétation de la formule.
Vous disposez toujours de la fonction `decompose(formule: str)-> (str, list[str])` et vous pouvez faire l'hypothèse que la valuation est correcte, c'est-à-dire qu'elle définit bien une valeur pour toutes les formules atomiques de la formule.

Exercice 3

Nous voulons maintenant construire la table de vérité d'une formule. Cela suppose :

- de construire toutes les interprétations possibles de ses formules atomiques,
- d'interpréter la formule pour chacune de ces interprétations.

Pour construire l'ensemble des interprétations, nous allons commencer par construire l'ensemble des parties de l'ensemble des formules atomiques, puis associer une interprétation v_P à chaque partie P selon la règle suivante :

si la formule atomique q appartient à P, alors $v_P(q) =$ `True` sinon $v_P(q) =$ `False`

Par exemple, pour une formule F dont l'ensemble des formules atomiques est $FA_F =$ `{'p', 'q'}`, l'ensemble des parties de FA_F est {$\emptyset$, `{'p'}`, `{'q'}`, `{'p', 'q'}`}. Les valuations correspondantes sont

- `{'p': False, 'q': False}` pour $P = \emptyset$
- `{'p': True, 'q': False}` pour $P =$ `{'p'}`
- `{'p': False, 'q': True}` pour $P =$ `{'q'}`
- `{'p': True, 'q': True}` pour $P =$ `{'p', 'q'}`

Question 1

Considérons d'abord le calcul de l'ensemble des parties de E.

Soit $E = \{x_i\}_{i \in 0..n}$ et la suite définie par $E_0 = \emptyset$ et $E_{i+1} = E_i \cup \{x_i\}$.

On peut alors calculer l'ensemble $\mathcal{P}(E)$ des parties de E de la manière suivante :

- $\mathcal{P}(E_0) = \{\emptyset\}$
- $\mathcal{P}(E_{i+1}) = \mathcal{P}(E_i) \cup \{A \cup \{x_i\} \mid A \in \mathcal{P}(E_i)\}$
- $\mathcal{P}(E) = \mathcal{P}(E_{n+1})$

Calculons par exemple les parties de $E = \{a, b\}$:

- $x_0 = a$ $(E_1 = \{a\})$. Donc $\mathcal{P}(E_1) = \{\emptyset\} \cup \{\emptyset \cup \{a\}\} = \{\emptyset, \{a\}\}$
- $x_1 = b$ $(E_2 = \{a, b\})$. Donc $\mathcal{P}(E_2) = \{\emptyset, \{a\}\} \cup \{\emptyset \cup \{b\}, \{a\} \cup \{b\}\}$

Donc $\mathcal{P}(E) = \mathcal{P}(E_2) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$

Écrivez une fonction `genere_sous_ensembles(ensemble : set)-> list[set]` qui prend en paramètre un ensemble et qui renvoie la liste composée de l'ensemble de ses parties.

Vous pourrez, si vous l'estimez utile, utiliser la fonction `deepcopy` du module `copy` qui permet de dupliquer un ensemble et ses éléments.

Question 2

Écrivez une fonction `genere_interpretations(fa : set)-> list[dict]` qui prend en paramètre un ensemble de formules atomiques et qui renvoie la liste composée de toutes les interprétations que l'on peut construire pour cet ensemble de formules.

Vous pourrez, si vous l'estimez utile, écrire d'abord une fonction qui construit une interprétation à partir d'un ensemble de formules atomiques et d'une partie de cet ensemble. Par exemple, `{'p': True, 'q': False}` pour $FA_F =$ `{'p', 'q'}` et $P =$ `{'p'}`.

Question 3

Écrivez une fonction `cree_table(formule : str)-> list[list]` qui prend en paramètre une formule et qui construit la table de vérité pour cette formule. La fonction insère dans la liste une première liste contenant les formules atomiques de la formule à évaluer et la formule elle-même. Les autres éléments associent à chaque valuation des formules atomiques l'interprétation correspondante pour la formule.

Par exemple, `cree_table("p AND q")` renvoie :

```
[['p', 'q', 'p AND q'], [False, False, False], [True, False, False], [False, True, False], [True, True, True]]
```

Vous avez accès aux différentes fonctions écrites jusqu'à présent :

- `genere_fa(formule : str)-> set[str]`
- `eval_formule(formule : str, valuation : dict)-> bool`
- `genere_interpretations(fa : set)-> list[dict]`

Exercice 4

Pour écrire les fonctions de cet exercice, vous pouvez utiliser les fonctions définies précédemment, à l'exception de celle qui construit la table de vérité puisqu'il est souvent inutile de calculer toutes les interprétations pour déterminer si une formule est satisfiable ou valide.

Question 1
Écrivez la fonction `is_satisfiable(formule : str)-> bool` qui renvoie `True` si la formule en paramètre est satisfiable, `False` sinon.

Question 2
Écrivez la fonction `is_valid(formule : str)-> bool` qui renvoie `True` si la formule en paramètre est valide, `False` sinon.

Question 3
Écrivez la fonction `models(formule : str)-> list[dict]` qui prend en paramètre une formule et qui construit la liste des modèles de cette formule, i.e., des interprétations pour lesquelles la formule s'interprète à `True`.

Par exemple, `models("p OR q")` renvoie :

```
[{'p': False, 'q': True}, {'p': True, 'q': False}, {'p': True, 'q': True}]
```

Une formule F_2 est la conséquence sémantique d'une formule F_1, noté $F_1 \models F_2$, si et seulement si pour toute interprétation I telle que $[F_1]^I = 1$, on a $[F_2]^I = 1$.

Question 4
Écrivez la fonction `consequence_sem(formule1 : str, formule2 : str)-> bool` qui renvoie `True` si `formule2` est une conséquence sémantique de `formule1`, `False` sinon.

Exercice 5

Il est temps maintenant de s'intéresser à la boîte à outils.

La fonction `decompose(formule : str)-> (str, list[str])` renvoie le couple formé du connecteur principal de la formule (la racine de l'arbre de syntaxe abstraite) et de la liste de ses sous-formules. Nous allons l'écrire en deux temps : décomposition d'une formule simple (au plus un connecteur) puis décomposition d'une formule générale.

Pour écrire les fonctions de cet exercice, vous pourrez être amenés à utiliser les fonctions `find`, `split` et `strip` qui opèrent sur les chaînes de caractères Python.

Question 1
Écrivez une première version de la fonction `decompose` que nous appellerons `decompose_simple` et qui s'applique aux formules sans parenthèses qui contiennent donc au plus un connecteur.

Par exemple,

- `decompose_simple("p")` renvoie `('p', [])`,
- `decompose_simple("NOT p")` renvoie `('NOT', ['p'])`,
- `decompose_simple("p IMPLIES q")` renvoie `('IMPLIES', ['p', 'q'])`.

Vous pouvez faire l'hypothèse que la formule passée en paramètre est bien sans parenthèses.

Pour écrire la fonction `decompose` qui décompose une formule dans le cas général, nous allons nous intéresser à l'équilibre des parenthèses dans la formule.

- si le nombre de parenthèses ouvrantes ne correspond pas au nombre de parenthèses fermantes, la formule est mal parenthésée et la fonction renvoie `None` ;

Une fois que l'on s'est assuré qu'il y a bien autant de parenthèses ouvrantes que de parenthèses fermantes :

- si la formule commence par une parenthèse ouvrante et se termine par une parenthèse fermante, ces parenthèses sont inutiles, on reprend l'analyse en les supprimant ;
- sinon on recherche la première position `i` à partir de laquelle le nombre de parenthèses ouvrantes et fermantes s'équilibre. Le connecteur principal est le premier qui suit cette position.

Question 2

Écrivez a fonction `decompose(formule : str)-> (str, list[str])` qui renvoie le couple formé du connecteur principal de la formule et de la liste de ses sous-formules. La fonction `decompose` doit traiter tous les cas, y compris les cas simples (vous pouvez faire appel à la fonction `decompose_simple`). Vous disposez de la fonction

```
trouve_prochain_connec(formule : str, index : int)-> (str, int)
```

qui renvoie le premier connecteur apparaissant dans formule à partir de la position `index` et son indice dans `formule`. Par exemple, `trouve_prochain_connec("toto IMPLIES titi", 2)` renvoie `('IMPLIES', 5)`.

Question 3

Écrivez a fonction

```
trouve_prochain_connec(formule : str, index : int)-> (str, int)
```

qui renvoie le nom et la position dans formule du premier connecteur dont la position est au moins égale à `index`. Par exemple, `trouve_prochain_connec("(NOT b)AND (q OR r)", 2)` renvoie `('AND', 8)`.

Thème 4 - TP

Objectifs

- DPLL
- SAT

Exercices

Le TME utilise Glucose, un solveur SAT performant installé sur les machines de la PPTI. Il prend en argument un fichier texte au format DIMACS décrit ci-dessous, représentant un ensemble de formules propositionnelles mises sous forme normale conjonctive, selon la syntaxe

`glucose clauses.cnf`

`glucose` indique alors si l'ensemble de clauses est satisfiable ou non. Dans le premier cas, il indique un modèle, représenté au format dimacs.

Le format DIMACS est le format d'entrée des solveurs SAT en général et de `glucose` en particulier. Sa première ligne (hormis d'éventuelles lignes de commentaires, voir ci-dessous) est de la forme

`p cnf` n k

où n est le nombre de variables propositionnelles du problème et k le nombre de clauses. Chacune des k lignes suivantes (hormis d'éventuelles lignes de commentaires) représente une clause, avec la syntaxe suivante : chaque variable propositionnelle est représentée par un entier entre 1 et n, la négation de i est représentée par $-i$. Une clause est écrite comme la suite de ses littéraux (variable propositionnelle ou sa négation), séparés par des espaces, terminée par un 0.

Enfin, toute ligne commençant par `c` est considérée comme un commentaire et ignorée par le solveur.

Exemple : on peut considérer le fichier `exemple.cnf` suivant

```
c Petit exemple illustratif
c
p cnf 5 3
-5 4 0
-1 5 3 4 0
-3 -4 0
```

Les deux premières lignes sont des commentaires. La troisième ligne indique que le problème considéré fait intervenir 5 variables et 3 clauses, représentées dans les trois lignes qui suivent. Ainsi cet exemple code la formule $(\neg x_5 \vee x_4) \wedge (\neg x_1 \vee x_5 \vee x_3 \vee x_4) \wedge (\neg x_3 \vee \neg x_4)$

On peut noter que la variable x_2 n'apparaît pas dans la formule et n'intervient donc pas sur sa satisfiabilité, mais elle est supposée exister. `glucose` répond alors

```
s SATISFIABLE
v -1 -2 -3 -4 -5 0
```

qui indique que la formule est satisfiable et que l'interprétation où toutes les variables sont à faux (parce qu'elles sont toutes précédées du symbole $-$) en est un modèle.

Exercice 6 – Prise en main de glucose et DIMACS

On considère l'ensemble de clauses du premier exercice de la feuille de TD, rappelé ici :

$$C_1 = a \vee b \quad C_2 = \neg a \vee b \vee c \quad C_3 = b \vee \neg c \vee d \quad C_4 = \neg b \vee \neg e \quad C_5 = e \vee d \quad C_6 = \neg d$$

Question 1
Ecrire cette théorie clausale au format DIMACS.

Question 2
Appliquer glucose et interpréter les résultats renvoyés en les comparant aux résultats établis en TD.

Question 3
Mêmes questions quand on remplace C_3 par $D_3 : \neg b \vee \neg c \vee d$.

Question 4
Utiliser `glucose` pour montrer que l'ensemble de clauses $\{p \vee q; \neg p \vee q; p \vee \neg q; \neg p \vee \neg q\}$ est insatisfiable.

Question 5
Utiliser `glucose` pour trouver un modèle de l'ensemble de clauses $\{a \vee b \vee c; \neg a \vee \neg b; \neg b \vee \neg c; \neg c \vee \neg a\}$

Question 6
Comment obtenir un autre modèle ? (indication : nier la solution obtenue)

Exercice 7 – Pizzas et glucose

On cherche maintenant à résoudre l'exercice sur le choix d'une pizza en utilisant glucose.

Rappel : trois amis, Laure, Marie et Nicolas, veulent commander une pizza à se partager. La pizzeria propose des pizzas utilisant 4 sortes de garnitures : des champignons, des poivrons, du jambon ou de l'œuf.

Marie pose un veto sur toute pizza avec des champignons. Nicolas ne veut pas d'œuf et veut qu'il y ait du jambon dans la pizza. Laure, elle, n'apprécie le jambon que s'il est accompagné de champignons, elle s'oppose donc aux pizza contenant du jambon sans contenir de champignons.

Question 1
En incluant des commentaires expliquant la représentation choisie, écrire un fichier DIMACS permettant de répondre à la question "Peut-on composer une pizza qui satisfasse tout le monde ?"

Question 2
Même question pour le problème où l'on cherche une pizza qui satisfasse au moins 2 personnes, en identifiant toutes les possibilités.

Exercice 8 – Club écossais

Un club écossais possède le règlement suivant :
Article premier : tout membre non écossais porte des chaussettes à pois
Article deuxième : tout membre porte un kilt ou ne porte pas de chaussettes à pois
Article troisième : les membres mariés ne sortent pas le dimanche
Article quatrième : un membre sort le dimanche si et seulement si il est écossais
Article cinquième : tout membre qui porte un kilt est écossais et marié
Article sixième : tout membre écossais porte un kilt

Question 1
Formaliser ce problème, le mettre sous forme normale conjonctive et utiliser `glucose` pour montrer que ce club n'a pas de membre.

Question 2
On supprime le sixième article. Utiliser `glucose` pour caractériser les membres du club.

Exercice 9 – Qui porte un chapeau rouge

Alice, Bob, Chloé et David portent un chapeau soit rouge, soit bleu. Les quatre affirmations suivantes sont vérifiées :
- Alice et Bob portent un chapeau de la même couleur
- Chloé et Bob portent des chapeaux de couleur différente
- David porte un chapeau rouge
- Ils sont au plus deux à porter un chapeau rouge

Question 1

Formaliser ce problème, le mettre sous forme normale conjonctive et utiliser `glucose` pour montrer que David et Chloé portent un chapeau rouge et Alice et Bob un chapeau bleu. Vous devez vous assurer que c'est la seule solution possible.

Thème 5 - TP

Objectifs

- SAT, glucose et dimacs

Exercices

L'objectif de ce TP est d'illustrer un cas d'application du formalisme DPLL-SAT et du logiciel glucose, combiné à un programme en python, pour un problème de répartition de tâches, dans un cadre de gestion de crises de type catastrophe naturelle.

Contexte On considère que la gestion de la situation de crise requiert de résoudre simultanément un certain nombre de ***tâches***, comme d'évaluer la zone (observer les dégâts, les risques), distribuer des vivres et de l'eau, déblayer un accès bloqué ou réparer des matériels critiques.

Chaque tâche requiert des ***compétences*** spécifiques : par exemple, pour déblayer un accès bloqué, il faut disposer de compétences en logistique (gestion de matériel), sécurité (protection du périmètre) et coordination (communication et gestion d'équipe).

L'équipe de gestion de la crise contient divers ***agents***, qui possèdent chacun des compétences propres : ainsi, une infirmière a des compétences en soins et premiers secours, une cheffe d'équipe a des compétences en logistique, coordination et sécurité.

L'annexe 1 donne un exemple, appelé `crise_0` dans la suite, contenant 5 compétences, 8 agents et 6 tâches, considéré tout au long du TP.

Objectif général Etant donné un ensemble de tâches associées à des ensembles de compétences requises et un ensemble d'agents associés à des ensembles de compétences, on cherche une affectation d'agents (potentiellement plusieurs agents) à chaque tâche pour que toutes les tâches puissent être effectuées. Un agent ne peut être affecté à plusieurs tâches simultanément.

Avec l'exemple `crise_0`, si on doit résoudre seulement les tâches d'identifiant 0 et 1, on peut affecter Samir à la tâche 0, mais il faut un binôme pour la tâche 1, parce qu'aucun agent ne possède les trois compétences requises. On peut par exemple affecter à la tâche 1, Hugo et Clara, ou Hugo et Léa, ou Nadia et Léa, ou Nadia et Julien, ou... Le problème est facile pour ces 2 tâches, mais il est plus complexe pour les 6 tâches.

Formalisation et objectif du TP Le problème de répartition des tâches peut être représenté comme la satisfiabilité d'un ensemble de formules logiques (à écrire et mettre sous forme clausale) et résolu par l'utilisation du logiciel glucose (après traduction au format dimacs). C'est donc une application des séances de TD et TP précédentes.

Toutefois, on ne peut pas écrire à la main le fichier dimacs contenant la traduction de toutes les formules logiques : la représentation informatique appropriée n'est pas lisible et le nombre de clauses à gérer est trop important.

L'objectif du TP est donc d'écrire un programme en python permettant de générer ce fichier dimacs approprié. Le programme doit être générique, c'est-à-dire prendre en arguments des tâches, des compétences et des agents quelconques. Les fonctions pourront être testées avec l'exemple `crise_0` décrit dans l'annexe 1, pour lequel des éléments de représentation en python sont fournis dans le fichier `gestion_crise.py` disponible sur le moodle et décrit dans la suite. Ce fichier doit être complété au cours du TP.

Le TP contient plusieurs exercices, visant à prendre en main la modélisation informatique du problème, résoudre le problème de la modélisation logique et écrire les clauses nécessaires, puis les traduire en un fichier dimacs. Il faut enfin pouvoir décoder le résultat renvoyé par glucose.

NB Des **rappels** de fonctions utiles en python sont fournis dans l'annexe 2, qui contient également un rappel sur le format dimacs et l'appel au logiciel glucose.

Exercice 10 – Modélisation informatique du problème

On considère que les compétences, les agents et les tâches sont représentés par des identifiants entiers (voir la première colonne des tableaux fournis en annexe pour l'exemple). La correspondance avec les noms est représentée dans trois dictionnaires de type `dict[int,str]` pour les compétences, les agents et les tâches respectivement.

Ces correspondances pour l'exemple `crise_0` sont fournies dans les dictionnaires `D_comp_0`, `D_agents_0` et `D_taches_0` dans le le fichier `gestion_crise.py`.

De plus, on utilise les représentations suivantes :

- chaque agent est représenté comme un couple de type `tuple[int, set[int]]`, où l'entier est son identifiant et l'ensemble contient les identifiants des compétences qu'il possède.
- chaque tâche est représentée comme un couple de type `tuple[int, set[int]]`, où l'entier est son identifiant et l'ensemble contient les identifiants des compétences que la tâche requiert.

Le fichier `gestion_crise.py` contient les listes `L_agents_0` et `L_taches_0` qui représentent respectivement tous les agents et toutes les tâches considérés pour l'exemple `crise_0`.

Question 1

Ecrire une fonction `competences_agent` qui, étant donné l'identifiant d'un agent, une liste d'agents et un dictionnaire de compétences, renvoie l'ensemble des chaînes de caractères correspondant aux compétences qu'il possède.

Ainsi, `competences_agent(5, L_agents_0, D_comp_0)` doit renvoyer l'ensemble
`{'sécurité', 'technique', 'coordination'}`
En effet, l'agent d'identifiant 5 est Samir, qui possède ces trois compétences. (on rappelle que dans un ensemble, l'ordre des éléments n'a pas d'importance, l'affichage peut donc différer).

Exercice 11 – Modélisation logique du problème

Pour modéliser le problème, on note nb_T le nombre total de tâches considérées et nb_A le nombre total d'agents dans l'équipe de gestion de la crise.

On introduit les variables $att_{t,a}$ où t et a sont des entiers tels que $0 \leq t < nb_T$ et $0 \leq t < nb_A$. Si une interprétation donne à $att_{t,a}$ la valeur 1, cela signifie que l'agent d'identifiant a est affecté à la tâche d'identifiant t.

Question 1

Clauses représentant les affectations aux tâches

La première contrainte, nommée règle 1 dans la suite, est que chaque tâche doit avoir au moins un agent pour chaque compétence.

1. Donner la formule logique traduisant cette contrainte pour l'exemple `crise_0`, dans le cas particulier de la tâche d'évaluation (identifiant 0) et en considérant uniquement la compétence sécurité. Mettre la formule sous forme clausale.
2. Pour l'exemple `crise_0`, donner la formule logique pour la tâche d'évaluation en considérant toutes ses compétences et la mettre sous forme clausale.
3. Etant donné une liste de tâches générique, combien de clauses sont générées par cette règle 1 ?
   Calculer cette valeur pour l'exemple `crise_0`.
4. Quelle clause est générée dans le cas où la liste de tâches inclut une tâche additionnelle d'identifiant 6, de nom court réseau, requérant une compétence de nom court informatique, pour le même ensemble d'agents ?

Question 2

Clauses représentant les affectations des agents

La seconde contrainte, nommée règle 2 dans la suite, est que chaque agent doit être affecté à au plus une tâche.

Cette contrainte peut aussi être formulée comme le fait que pour chaque agent, pour chaque paire de tâches distinctes, un agent ne doit pas être affecté aux deux tâches.

1. On considère l'exemple `crise_0` et l'agent Léa. Donner la formule logique, sous forme clausale, correspondant à cette contrainte.
2. Etant donné une liste d'agents et une liste de tâches génériques, donner la formule logique représentant cette contrainte.
3. Indiquer le nombre de clauses générées par cette contrainte pour des listes de tâches et d'agents données, puis calculer cette valeur pour l'exemple `crise_0`.

Exercice 12 – Génération du fichier DIMACS

Attention, avant de poursuivre le TP assurez-vous auprès de votre enseignant ou enseignante que vous avez bien répondu aux deux questions précédentes.

On souhaite maintenant générer un fichier DIMACS qui sera passé en entrée de glucose (voir TP6 et une version résumée à la fin de l'annexe 2 de ce sujet).

Question 1

Codage et décodage des variables $att_{t,a}$

Pour être compatible avec le format DIMACS, on choisit de représenter chaque variable $att_{t,a}$, pour $0 \leq t < nb_T$ et $0 \leq a < nb_A$ par un entier, défini comme $nb_A \times t + a + 1$.

Ainsi, pour la tâche 3 et l'agent 2 parmi un nombre total de 8 agents, la fonction doit renvoyer 27.

Pour la tâche 5 et l'agent 1 parmi un nombre total de 8 agents, la fonction doit renvoyer 42.

1. Ecrire une fonction `codage` qui prend en arguments trois entiers naturels t, a et nb_A, tels que $0 \leq t$ et $0 \leq a < nb_A$, et qui renvoie l'entier qui encode la variable $att_{t,a}$.

   Par exemple `codage(3, 2, 8)` doit renvoyer 27.
2. Ecrire une fonction `decodage` qui prend en argument deux entiers n et nb_A et qui renvoie le couple (t, a) tel que n est l'entier qui encode la variable $att_{t,a}$.

   Indice : cette fonction peut utiliser avec profit une division entière par nb_A (Rappels python : l'opérateur `//` rend le quotient de la division entière, l'opérateur `%` rend le reste).

   Par exemple, `decodage(42, 8)` doit renvoyer (5, 1).

Question 2

Construction de dictionnaire auxiliaire

Ecrire une fonction `dico_comp` qui prend en argument une liste d'agents et qui renvoie un dictionnaire associant à chaque identifiant de compétence l'ensemble des identifiants d'agents possédant cette compétence.

Ainsi, pour l'exemple `crise_0`, `dico_comp(L_agents_0)` doit renvoyer le dictionnaire
`{0: {3, 6}, 1: {0, 1, 2, 7}, 2: {1, 2, 4, 5}, 3: {0, 4, 5}, 4: {0, 2, 5, 6, 7}}`
(On rappelle que dans un dictionnaire l'ordre des clés n'a pas d'importance, l'affichage peut donc différer)

Question 3

Codage des clauses représentant les affectations aux tâches

1. Ecrire une fonction `codage_regle_1` qui prend en arguments une liste de tâches et une liste d'agents et qui renvoie une liste de chaîne de caractères dont chaque élément représente, au format dimacs, l'une des clauses correspondant à la règle 1 (question 1 de l'exercice 2).

   Si on l'applique uniquement à la première tâche de l'exemple `crise_0` (la tâche d'évaluation), par l'appel `codage_regle_1([L_taches_0[0]], L_agents_0)`, on doit obtenir (l'ordre peut varier dans chacune des trois chaînes de caractères)
   `['2 3 5 6 0', '1 5 6 0', '1 3 6 7 8 0']`
2. Vérifier que la longueur de la liste obtenue pour l'exemple `crise_0` globalement, correspondant à l'appel `codage_regle_1(L_taches_0, L_agents_0)` , correspond au résultat attendu (voir Question 1 de l'exercice précédent).

3. Si besoin, modifier la fonction précédente pour gérer le cas particulier évoqué dans la question 1.4 de l'exercice 2.

Question 4

Codage des clauses représentant les affectations aux agents

Ecrire une fonction `codage_regle_2` qui prend en arguments une liste de tâches et une liste d'agents et qui renvoie une liste de chaînes de caractères dont chaque élément représente, au format dimacs, l'une des clauses correspondant à la règle 2 (question 2 de l'exercice 2).

Vérifier que la longueur de la liste obtenue pour l'exemple `crise_0`, correspondant à l'appel `codage_regle_2(L_taches_0, L_agents_0)` , correspond au résultat attendu (voir Question 2 de l'exercice précédent).

L'incompatibilité de l'affectation de Léa (identifiant 0) à la tâche d'évaluation (identifiant 0) avec d'autres tâches doit notamment conduire à la présence dans la liste des éléments suivants :

```
['-1 -9 0', '-1 -17 0', '-1 -25 0', '-1 -33 0', '-1 -41 0', '-9 -17 0']
```

Question 5

Ecriture du fichier dimacs

Ecrire une fonction `genere_dimacs` qui prend en argument une chaîne de caractères correspondant au nom de fichier de sauvegarde souhaité, une liste de tâches et une liste d'agents et qui génère le fichier dimacs correspondant.

Utiliser pour cela les fonctions précédentes ainsi que la fonction `sauvegarde_fichier` fournie dans le fichier `gestion_crise.py` du moodle. Celle-ci prend en argument une chaîne de caractères correspondant à un nom de fichier correct et une liste de chaînes de caractères. Elle écrit dans le fichier les chaînes de caractères sur des lignes séparées (chaque chaîne est suivie d'un retour à la ligne).

NB si le fichier existe déjà, son contenu est remplacé par cette liste de chaînes de caractères et donc perdu.

Question 6

Décodage

Utiliser glucose sur le fichier généré pour l'exemple `crise_0`.

La réponse attendue est que le problème est satisfiable et qu'il a pour modèle

```
v -1 -2 -3 -4 -5 6 -7 -8 -9 -10 -11 12 -13 -14 -15 16 -17 18 -19 -20 21 -22
-23 -24 -25 -26 27 -28 -29 -30 -31 -32 33 -34 -35 -36 -37 -38 -39 -40 -41 -42
-43 -44 -45 -46 47 -48 0
```

La dernière étape consiste à décoder ce résultat pour en déduire l'affectation des agents aux tâches : copier-coller à la souris le résultat ci-dessus, renvoyé par glucose, dans une chaîne de caractères appelée `res_crise_0` pour l'utiliser pour les tests.

1. Ecrire une fonction `affectation_agent` qui prend en argument une chaîne de caractères correspondant au modèle affiché par glucose dans le cas où le problème est satisfiable, ainsi qu'un dictionnaire d'agents et un dictionnaire de tâches, et qui renvoie un dictionnaire ayant pour clé les prénoms des agents et pour valeurs les noms des tâches auxquelles ils sont affectés.
2. Ecrire une fonction `affectation_taches` prenant les mêmes arguments et renvoyant un dictionnaire ayant pour clés les noms des tâches et pour valeurs l'ensemble des prénoms des agents affectés à cette tâche

   NB il est efficace de faire une fonction `affectation` ayant les mêmes arguments et renvoyant un tuple contenant les deux dictionnaires, pour faire un seul parcours de la chaîne de caractères.

Question 7

Variantes

1. Que se passe-t-il si ni Hugo ni Nadia ne font partie de l'équipe ?

   Utiliser les fonctions précédentes sur une liste d'agents modifiée, où ils sont absents, et examiner le résultat.
2. Compléter le programme pour que, dans le cas où une première affectation est trouvée, on puisse chercher si d'autres sont possibles.

Annexes

Annexe 1 : Exemple

Compétences considérées

identifiant	nom court	descriptif
0	secours	soins, premiers secours
1	logistique	logistique, gestion du matériel
2	technique	maintenance, réparations, équipements
3	sécurité	sécurité, protection du périmètre
4	coordination	coordination, communication, gestion d'équipe

Agents considérés

identifiant	prénom	rôle	compétences (nom court)
0	Léa	chef d'équipe	logistique, sécurité, coordination
1	Julien	agent de maintenance	logistique, technique
2	Karim	logisticien	logistique, technique, coordination
3	Hugo	infirmier	secours
4	Zoé	technicienne	technique, sécurité
5	Samir	sécurité terrain	technique, sécurité, coordination
6	Nadia	médecin urgentiste	secours, coordination
7	Clara	chargée de communication	logistique, coordination

Tâches considérées

Identifiant	nom court	compétences requises	description
0	évaluation	sécurité, technique, coordination	observer les dégâts, risques, accès praticables
1	poste médical	secours, logistique, coordination	monter les tentes, organiser un espace de soins
2	déblayage	sécurité, technique, logistique	déblayer un accès bloqué
3	générateur et radio	technique, logistique, coordination	diagnostiquer et réparer des matériels critiques
4	distribution vivres	logistique, sécurité, coordination	gérer les stocks, flux et files d'attentes
5	premiers soins	secours, coordination	stabiliser les blessés

Annexe 2 : Quelques rappels de python

Manipulations de listes

`[]`	création d'une liste vide
`[e1, e1]`	création d'une liste à 2 éléments
`len(L)`	taille de la liste `L`
`for e in L:`	parcours de la liste par élément
	`e` prend successivement pour valeur chacun des éléments de `L`
`for i in range(len(L)):`	parcours de la liste par position
`  e = L[i]`	`e` prend successivement pour valeur chacun des éléments de `L`
`L1 + L2`	concaténation des deux listes
`L.append(x)`	ajoute l'élément `x` à la fin de la liste `L`

Manipulations d'ensembles

`set()`	création d'un ensemble vide
`{e1, e1}`	création d'un ensemble à 2 éléments
`len(S)`	cardinal de l'ensemble `S`
`e in S`	test d'appartenance $e \in S$
`for e in S:`	parcours de l'ensemble
`S1 & S2`	intersection $S1 \cap S2$
`S1 | S2`	union $S1 \cup S2$
`S1 <= S2`	test d'inclusion $S1 \subseteq S2$
`S.add(x)`	ajoute x à S

Manipulations de dictionnaires

`dict()`	création d'un dictionnaire vide
`{k1:v1, k2:v2}`	dictionnaire à 2 entrées
`len(D)`	nombre de clés
`k in D`	test d'appartenance des clés
`for k in D:`	parcours des clés
`D[k]`	valeur associée à la clé `k`
`D[k] = v`	ajoute l'association `k:v` à `D` (écrase l'ancienne association si `k` déjà dans `D`)

Glucose (voir Thème 4)
Glucose est un solveur SAT qui prend en argument un fichier texte au format DIMACS, représentant un ensemble de formules propositionnelles mises sous forme normale conjonctive, selon la syntaxe
`glucose clauses.cnf`
`glucose` indique alors si l'ensemble de clauses est satisfiable ou non. Dans le premier cas, il indique un modèle, représenté au format dimacs.

Format DIMACS (voir Thème 4)
Un fichier DIMACS a la structure suivante : sa première ligne du fichier est de la forme
`p cnf` n k
où n est le nombre de variables propositionnelles du problème et k le nombre de clauses.

Chacune des clauses est représentée sur une ligne avec la syntaxe suivante : chaque variable propositionnelle est représentée par un entier entre 1 et n, la négation de i est représentée par $-i$. Une clause est écrite comme la suite de ses littéraux (variable propositionnelle ou sa négation), séparés par des espaces, terminée par un 0. Par exemple
`1 -5 3 -4 0`
représente la clause $x_1 \vee \neg x_5 \vee x_3 \vee \neg x_4$

Toute ligne commençant par `c` est considérée comme un commentaire et ignorée par le solveur.

Thème 6.7 - TP

Objectifs

- Décuction naturelle

Exercices

Preuves sur Edukera, accès sur moodle.

Thème 8 - TP

Objectifs

- Logique des prédicats du premier ordre
  - termes
  - syntaxe
  - sémantique
  - interprétation

Exercices

Les exercices sur les échiquiers proposés sur la plateforme Edukera reposent sur un mécanisme permettant de vérifier si des propriétés sur les pièces placées sur un échiquier exprimées par une formule logique sont satisfaites par un échiquier donné. Par exemple, avec l'échiquier de la figure 1, la formule exprimant que tous les pions sont sur la même ligne est "vraie" tandis que la formule exprimant que le fou est de la même couleur que le cavalier est "fausse". Il est ici possible de déplacer les pièces de cet échiquier pour changer le résultat de l'interprétation de ces deux formules.

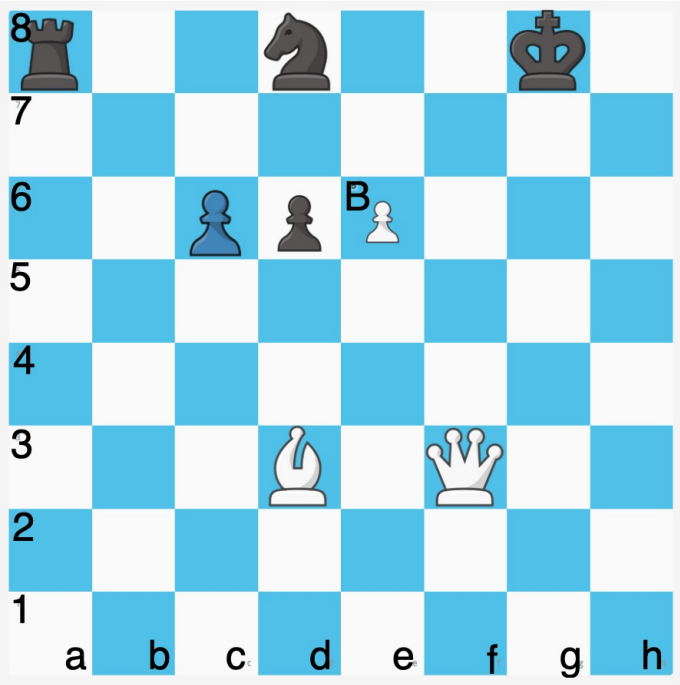


FIGURE 1 – Exemple d'échiquier de la plateforme Edukera

L'exercice 1 identifie les questions que vous devez faire sur Edukera. Les exercices 2 et 3 vous permettent de faire le lien entre les échiquiers et les structures.

Sur Edukera, pour avoir accès aux "boutons" permettant de changer la taille des pièces vous devez passer en thème sombre (voir figure 2)



FIGURE 2 – Choix du thème sombre

Exercice 13 – Exercices Edukera

L'intérêt des exercices ne réside pas uniquement dans l'élaboration d'une solution. Il est tout aussi important de comprendre pourquoi une solution est correcte que de comprendre ce qui la rendrait fausse. Pour que vos solutions aient du sens, vous devez :

- toujours poser sur l'échiquier au moins une pièce de chaque type apparaissant dans la formule (roi, reine, ...)
- une fois que vous avez trouvé une configuration satisfaisant la formule, bougez, modifiez les pièces pour bien comprendre pouquoi la formule est satisfaite

Question 1

Échiquier 1 - compréhension des prédicats portant sur la taille et la position

1. plusGrand
2. entre
3. droiteDe
4. basDe
5. hautDe
6. gaucheDe
7. ensemble des prédicats

Pour les échiquiers suivants, il vous est précisé, pour chaque question, les pièces que vous devez au minimum poser sur le plateau.

Question 2

Échiquier 4 - formules avec quantificateurs

8. deux reines de tailles différentes
9. deux reines de tailles différentes
10. deux pions de tailles différentes
11. deux rois de tailles différentes
12. deux pions et une autre pièce sur le plateau. Que se passe-t-il s'il n'y a aucun pion sur le plateau ?
13. deux rois et une autre pièce sur le plateau. Que se passe-t-il s'il n'y a aucun roi sur le plateau ?
14. deux reines et une autre pièce sur le plateau. Que se passe-t-il s'il n'y a aucune reine sur le plateau ?
15. un reine et un pion. Pouvez-vous trouver un plateau qui ne satisfait pas la formule ?
16. deux reines, deux pions et deux rois avec le plus de tailles différentes possibles.
17. deux petites, deux moyennes et deux grandes pièces de types les plus variés possibles
18. trois reines, trois pions et trois rois de tailles les plus variées possibles

Question 3
Échiquier 5 - combinaison de quantificateurs

19. Est-il possible de ne pas satisfaire la première formule ?
20. 3 pièces quelconques. A partir de combien de pièces n'est-il plus possible de satisfaire la formule ?

Question 4
Échiquier 6 - formalisation
Dans cette question, vous devez écrire la formule logique correspondant à la phrase donnée et placer des pièces pour que cette formule soit satisfaite par le plateau.

Question 5
Échiquier 9 - formalisation
Dans cette question, vous devez écrire la formule logique correspondant à la phrase donnée et placer des pièces pour que cette formule soit satisfaite par le plateau. Pour la question 30, la formule obtenue se décompose en deux parties, une partie exprimant qu'il n'y a pas de pion et l'autre partie exprimant qu'il y a un pion et donnant la condition à vérifier dans ce cas.

Question 6
Échiquier 10 - combinaison de quantificateurs

31. 32. 33. 34. deux pièces
35. 36. deux rois
37. les contraintes imposées par les formules

Exercice 14 – Syntaxe : définition de l'ensemble des termes

L'ensemble $\mathcal{T}(X, \mathcal{F})$ des termes considérés contient uniquement des constantes et des variables (la signature utilisée ne contient aucun symbole de fonction d'arité strictement positive) désignant des pièces sur un échiquier. Ces termes sont construits à partir d'un ensemble $\mathcal{F}_0$ contenant 8 symboles de constante (les 8 premières lettres de l'alphabet en majuscule) et d'un ensemble X contenant 26 symboles de variable (les lettres de l'alphabet en minuscule).

$$\mathcal{F} = \mathcal{F}_0 = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}, \mathrm{G}, \mathrm{H}\}$$
$$X = \{\mathrm{a}, \mathrm{b}, \mathrm{c}, \mathrm{d}, \mathrm{e}, \mathrm{f}, \mathrm{g}, \mathrm{h}, \mathrm{i}, \mathrm{j}, \mathrm{k}, \mathrm{l}, \mathrm{m}, \mathrm{n}, \mathrm{o}, \mathrm{p}, \mathrm{q}, \mathrm{r}, \mathrm{s}, \mathrm{t}, \mathrm{u}, \mathrm{v}, \mathrm{w}, \mathrm{x}, \mathrm{y}, \mathrm{z}\}$$

Les symboles de constante vont servir à nommer des pièces sur l'échiquier.

Interprétation des termes : construction d'un échiquier

Les termes désignent des pièces positionnées sur un échiquier contenant 8 colonnes (les abscisses de gauche à droite sont désignées par les lettres a, b, c, d, e, f, g et h) et 8 lignes (les ordonnées du bas vers le haut sont désignées par les entiers 1, 2, 3, 4, 5, 6, 7 et 8). Par exemple, le pion nommé B sur l'échiquier de la figure 1 se trouve sur la case de coordonnées (e,6).

A chaque pièce placée sur l'échiquier, on associe une position, une espèce, une couleur et une taille (il y a au plus une pièce par case de l'échiquier). L'ensemble Pos des positions possibles contient l'ensemble des coordonnées de

l'échiquier :

$$\text{Pos} = \left\{ \begin{array}{cccccccc} (\text{a},8) & (\text{b},8) & (\text{c},8) & (\text{d},8) & (\text{e},8) & (\text{f},8) & (\text{g},8) & (\text{h},8) \\ (\text{a},7) & (\text{b},7) & (\text{c},7) & (\text{d},7) & (\text{e},7) & (\text{f},7) & (\text{g},7) & (\text{h},7) \\ (\text{a},6) & (\text{b},6) & (\text{c},6) & (\text{d},6) & (\text{e},6) & (\text{f},6) & (\text{g},6) & (\text{h},6) \\ (\text{a},5) & (\text{b},5) & (\text{c},5) & (\text{d},5) & (\text{e},5) & (\text{f},5) & (\text{g},5) & (\text{h},5) \\ (\text{a},4) & (\text{b},4) & (\text{c},4) & (\text{d},4) & (\text{e},4) & (\text{f},4) & (\text{g},4) & (\text{h},4) \\ (\text{a},3) & (\text{b},3) & (\text{c},3) & (\text{d},3) & (\text{e},3) & (\text{f},3) & (\text{g},3) & (\text{h},3) \\ (\text{a},2) & (\text{b},2) & (\text{c},2) & (\text{d},2) & (\text{e},2) & (\text{f},2) & (\text{g},2) & (\text{h},2) \\ (\text{a},1) & (\text{b},1) & (\text{c},1) & (\text{d},1) & (\text{e},1) & (\text{f},1) & (\text{g},1) & (\text{h},1) \end{array} \right\}$$

Chaque pièce appartient à une espèce (les espèces sont présentées sur la figure 3), est d'une certaine taille (grande, moyenne ou petite) et d'une certaine couleur (blanche, noire, bleue). Les ensembles d'espèces, de tailles et de couleurs sont définis par :

$$\text{Esp} = \{\text{e_roi}, \text{e_reine}, \text{e_tour}, \text{e_fou}, \text{e_cavalier}, \text{e_pion}\}$$
$$\text{Tailles} = \{\text{t_petit}, \text{t_moyen}, \text{t_grand}\}$$
$$\text{Couleurs} = \{\text{c_blanc}, \text{c_noir}, \text{c_bleu}\}$$



e_roi e_reine e_tour e_fou e_cavalier e_pion

FIGURE 3 – Espèces des pièces de l'échiquier de la plateforme Edukera

Certaines pièces ont un nom : il s'agit d'une constante de $\mathcal{F}_0$. Par convention la valeur `None` sert à nommer les pièces anonymes (i.e. sans nom). Deux pièces différentes ne peuvent pas avoir le même nom lorsque ce nom est un élément de $\mathcal{F}_0$. Autrement dit tous les symboles de $\mathcal{F}_0$ ne servent pas nécessairement à nommer une pièce de l'échiquier et un nom de $\mathcal{F}_0$ ne peut pas servir à nommer deux pièces différentes. Un échiquier E est la donnée d'une grille sur laquelle sont disposées des pièces et est représenté par un ensemble de quintuplets. Chaque quintuplet $((x, y), e, t, c, n) \in E$ est un élément du produit cartésien :

$$\text{Pos} \times \text{Esp} \times \text{Tailles} \times \text{Couleurs} \times (\mathcal{F}_0 \cup \{\texttt{None}\})$$

et exprime qu'une pièce de nom n ($n =$ `None` si la pièce n'a pas de nom), d'espèce e, de taille t et de couleur c se trouve à la position (x, y) sur l'échiquier E. L'ensemble des échiquiers possibles est donc :

$$\mathbf{E} = \wp(\text{Pos} \times \text{Esp} \times \text{Tailles} \times \text{Couleurs} \times (\mathcal{F}_0 \cup \{\texttt{None}\}))$$

où lorsque S est un ensemble, $\wp(S)$ désigne l'ensemble des parties de S.

Question 1

Donner les éléments de l'ensemble E correspondant à l'échiquier représenté sur la figure 1 (sur cette figure, les pièces qui ne sont pas des pions sont toutes grandes et seul le petit pion blanc a un nom qui est B).

Question 2

Interprétation des termes : construction d'une structure à partir d'un échiquier

Etant donné un échiquier $E \in \mathbf{E}$, on construit une structure $\mathbf{M}_E$ permettant d'interpréter l'ensemble de termes $\mathcal{T}(X, \mathcal{F})$. La structure $\mathbf{M}_E$ est définie par :

- un domaine d'interprétation correspondant à l'ensemble de tous les quintuplets représentant les pièces positionnées sur l'échiquier E, auquel on ajoute un élément particulier (Error) :

$$|\mathbf{M}_E| = \{((x, y), e, t, c, n) \in E\} \cup \{\text{Error}\}$$

- une fonction d'interprétation qui associe un élément de $|\mathbf{M}_E|$ à chaque symbole de constante de $\mathcal{F}_0$ désignant une pièce de l'échiquier E : il s'agit du quintuplet correspondant à cette pièce sur l'échiquier (si aucune pièce de nom n se trouve sur l'échiquier, l'interprétation de n déclenche une erreur qui peut être vue comme une valeur particulière Error du domaine d'interprétation) :

$$n^{\mathbf{M}_E} = \begin{cases} ((x,y),e,t,c,n) & \text{si } ((x,y),e,t,c,n) \in E \\ \mathrm{Error} & \text{sinon} \end{cases}$$

Question 3
On considère à nouveau l'échiquier représenté sur la figure 1. Quelle est la valeur de $\mathrm{A}^{\mathbf{M}_E}$? de $\mathrm{B}^{\mathbf{M}_E}$?

Exercice 15 – Formules logiques

Symboles de prédicat

Les formules logiques de $\mathbb{F}(X,\mathcal{F},\mathcal{P})$ expriment des propriétés sur les pièces d'un échiquier. Elles sont construites à partir de l'ensemble X de symboles de variable, de l'ensemble $\mathcal{F} = \mathcal{F}_0$ de symboles de constante et de l'ensemble $\mathcal{P} = \mathcal{P}_1 \cup \mathcal{P}_2 \cup \mathcal{P}_3$ de symboles de prédicat où :

$$\begin{aligned} \mathcal{P}_1 &= \{\mathrm{roi}, \mathrm{reine}, \mathrm{tour}, \mathrm{fou}, \mathrm{cavalier}, \mathrm{pion}, \mathrm{blanc}, \mathrm{noir}, \mathrm{bleu}, \mathrm{petit}, \mathrm{moyen}, \mathrm{grand}\} \\ \mathcal{P}_2 &= \left\{ \begin{array}{l} \mathrm{droiteDe}, \mathrm{gaucheDe}, \mathrm{basDe}, \mathrm{hautDe}, \mathrm{plusPetit}, \mathrm{plusGrand}, \\ \mathrm{idLigne}, \mathrm{idColonne}, \mathrm{idTaille}, \mathrm{idCouleur}, = \end{array} \right\} \\ \mathcal{P}_3 &= \{\mathrm{entre}\} \end{aligned}$$

Interprétation des prédicats : construction d'une structure à partir d'un échiquier (suite)

Etant donné un échiquier E, l'interprétation d'un symbole de prédicat $p \in \mathcal{P}_k$ ($k \in \{1,2,3,\}$) est un ensemble de k-uplets de valeurs du domaine d'interprétation, c'est-à-dire un ensemble de k-uplets de quintuplets correspondant aux pièces placées sur l'échiquier. L'interprétation des prédicats d'espèce, de couleur et de taille de $\mathcal{P}_1$ est définie en examinant les quintuplets de E et en considérant la propriété souhaitée. Par exemple, l'interprétation du prédicat pion avec la structure $|\mathbf{M}_E|$ est définie par :

$$\mathrm{pion}^{\mathbf{M}_E} = \{((x,y),\mathrm{e_pion},t,c,n) \in E\}$$

Si E est l'échiquier représenté sur la figure 1, on a donc :

$$\mathrm{pion}^{\mathbf{M}_E} = \left\{ \begin{array}{l} ((\mathrm{c},6),\mathrm{e_pion},\mathrm{c_bleu},\mathrm{t_grand},\mathtt{None}),((\mathrm{d},6),\mathrm{e_pion},\mathrm{c_noir},\mathrm{t_moyen},\mathtt{None}), \\ ((\mathrm{e},6),\mathrm{e_pion},\mathrm{c_blanc},\mathrm{t_petit},\mathrm{B}) \end{array} \right\}$$

L'interprétation des prédicats de $\mathcal{P}_2$ et $\mathcal{P}_3$ est similaire mais nécessite des opérations de comparaison sur les composants des quintuplets de E. Par exemple, l'interprétation du prédicat $\mathrm{plusPetit}$ avec la structure $|\mathbf{M}_E|$ est définie par :

$$\mathrm{plusPetit}^{\mathbf{M}_E} = \left\{ \begin{array}{l} (((x_1,y_1),e_1,t_1,c_1,n_1),((x_2,y_2),e_2,t_2,c_2,n_2)) \in E \times E \\ \mid \left(\begin{array}{ll} & (t_1 = \mathrm{t_petit} \text{ et } (t_2 = \mathrm{t_moyen} \text{ ou } t_2 = \mathrm{t_grand})) \\ \text{ou} & (t_1 = \mathrm{t_moyen} \text{ et } t_2 = \mathrm{t_grand}) \end{array} \right) \end{array} \right\}$$

Si E est l'échiquier représenté sur la figure 1, on a donc :

$$\text{plusPetit}^{\mathbf{M}_E} = \left\{ \begin{array}{l} (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{c},6),\text{e_pion},\text{c_bleu},\text{t_grand},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{d},3),\text{e_fou},\text{c_blanc},\text{t_grand},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{f},3),\text{e_reine},\text{c_blanc},\text{t_grand},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{a},8),\text{e_tour},\text{c_noir},\text{t_grand},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{d},8),\text{e_cavalier},\text{c_noir},\text{t_grand},\texttt{None})), \\ (((\text{e},6),\text{e_pion},\text{c_blanc},\text{t_petit},\text{B}),((\text{g},8),\text{e_roi},\text{c_noir},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{c},6),\text{e_pion},\text{c_bleu},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{d},3),\text{e_fou},\text{c_blanc},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{f},3),\text{e_reine},\text{c_blanc},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{a},8),\text{e_tour},\text{c_noir},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{d},8),\text{e_cavalier},\text{c_noir},\text{t_grand},\texttt{None})), \\ (((\text{d},6),\text{e_pion},\text{c_noir},\text{t_moyen},\texttt{None}),((\text{g},8),\text{e_roi},\text{c_noir},\text{t_grand},\texttt{None})) \end{array} \right\}$$

Question 1
Etant donné un échiquier E, définir $p^{\mathbf{M}_E}$ pour tous les symboles de prédicat de $\mathcal{P}$.

Question 2
On considère à nouveau l'échiquier E représenté sur la figure 1. Donner les éléments de l'ensemble $p^{\mathbf{M}_E}$ pour tous les symboles de prédicat de $\mathcal{P}$.

Thème 9 - TP

Objectifs

- Déduction naturelle : logique des prédicats du premier ordre

Exercices

Preuves sur Edukera, accès sur moodle.

Thème 10 - TP

Objectifs

- Programmation prolog

Quelques rappels de syntaxe prolog

Eléments de base Prolog est un langage de programmation fondé sur la logique des prédicats du premier ordre.
- Les constantes sont représentées par des chaînes de caractères qui commencent par une minuscule.
- Les variables sont représentées par des chaînes de caractères qui commencent par une Majuscule.
- Les prédicats sont représentés par des chaînes de caractères qui commencent par une minuscule.

Ainsi, dans la proposition `frere(X,charlemagne)`, le prédicat est `frere`, d'arité 2 : il est appliqué à la variable `X` et à la constante `charlemagne`.

Les règles sont de la forme `prop :- prop1, prop2, prop3.` qui correspond à la formule de LPPO $(prop1 \wedge prop2 \wedge prop3) \rightarrow prop$.

Le prédicat à gauche de `:-` est la *tête* de la règle et les propositions à droite définissent le *corps* de la règle. Les propositions du corps sont séparées par des virgules.

Prolog repose sur la représentation clausale des formules : la formule
$((prop_1 \wedge prop_2 \wedge prop_3) \vee (prop_4 \wedge prop_5)) \rightarrow prop$, dont la forme normale conjonctive est
$(\neg prop_1 \vee \neg prop_2 \vee \neg prop_3 \vee prop) \wedge (\neg prop_4 \vee \neg prop_5 \vee prop)$, est représentée par les 2 règles

```
prop :- prop1, prop2, prop3.
prop :- prop4, prop5.
```

Chaque règle correspond à une clause comportant au plus un littéral positif.

Les faits correspondent aux clauses qui contiennent un littéral positif et aucun littéral négatif. Ils sont représentés par des règles dont le corps est vide, c'est-à-dire de la forme `prop.`

Programme Les faits et les règles sont écrits dans un fichier texte, dont le nom a pour extension `pl`. Il est préférable que le nom du fichier commence par une minuscule (cf ci-dessous la procédure de chargement du fichier).

Tout éditeur de texte peut être utilisé (par défaut, les fichiers d'extension `pl` sont considérés comme du `perl`, une coloration syntaxique pour prolog peut néanmoins être spécifiée. Par exemple avec `emacs`, on y accède avec alt+x prolog-mode).

Attention : il faut regrouper dans le fichier tous les faits et règles ayant la même tête, sinon l'interprète prolog provoque une erreur lors du chargement du fichier.

Le symbole `%` met en commentaire tout ce qui suit, sur la ligne où il figure.

Les symboles `/* */` permettent de mettre des commentaires plusieurs lignes.

L'interprète `swiprolog` Pour lancer l'interprète prolog, il faut taper dans un terminal la commande `swipl`. On entre en mode "requête", indiqué par l'invite `?-`

Toutes les commandes se terminent par un point. On peut en particulier utiliser
- `[nom].` ou `consult(nom).` ou `['nom.pl'].` : charge le fichier `nom.pl`.

  La commande `[nom].` ne convient pas si le nom du fichier commence par une majuscule : prolog considère que l'on fait référence à une variable et provoque une erreur. Elle ne convient pas non plus si le nom du fichier contient des caractères spécifiques (tiret, point, etc).

  A chaque nouveau chargement d'un fichier existant, les faits et les règles précédents sont écrasés par les nouvelles définitions.

- `help.` : affiche la fenêtre d'aide et la liste des prédicats prédéfinis dans SWI-Prolog.
- `help(pred).` : affiche l'aide concernant le prédicat `pred`.
- `trace.` : permet de suivre pas à pas l'évaluation des requêtes.
- `notrace.` et `nodebug.` : permettent de quitter le mode trace.
- `halt.` : quitte Prolog.

Requêtes Une fois le programme chargé, on peut entrer une requête, c'est-à-dire une proposition de la LPPO contenant des variables écrite avec la syntaxe prolog, qui se termine par "`.`".

Exercices

Exercice 16 – Premières requêtes

Question 1

Créer un fichier `exercice0.pl` contenant les deux faits

```
toto(a,b).
toto(c,b).
```

1. Le charger : l'affichage doit alors être de la forme

   ```
   ?- [exercice0].
   //1 compiled 0.01 sec, 624 bytes
   true.
   ```

2. Tester la requête `?- toto(X,b).` et comprendre les résultats obtenus.

   S'il n'y a qu'une instanciation possible, l'interprète rend la main. Sinon, il attend une saisie de l'utilisateur : la touche "Entrée" termine, la touche "`;`" donne successivement les autres solutions obtenue par exploration de l'arbre de résolution.

3. Même question avec `?- toto(b,X).`

4. Reprendre les deux exemples précédents en mode trace, pour observer les étapes d'unification effectuées

   NB prolog renomme toutes les variables avec des noms de la forme $_G2673$ ou un autre entier.

Question 2

Tester en mode trace les deux programmes du dernier exercice de TD sur l'arbre de résolution au format prolog.

Exercice 17 – Relations familiales

1. Télécharger le fichier `familleSimpson.pl` disponible sur le moodle et le charger dans prolog.

   Les prédicats binaires `pere(X, Y)` et `mere(X, Y)` indiquent que `X` est le père (respectivement la mère) de `Y`.

2. Dessiner sur papier l'arbre généalogique correspondant.

3. Pour chacune des requêtes suivantes, indiquer en français quelle est la question qu'elle représente, prédire les réponses attendues puis vérifier que prolog les donne bien.

   (a) `pere(clancy, X).`

   (b) `mere(X, homer).`

   (c) `mere(homer, X).`

(d) `mere(X, herb).`
(e) `pere(homer, X), feminin(X).`
(f) `feminin(X), pere(homer, X).`

Pour cette requête et la précédente, examiner l'arbre de résolution en utilisant le mode trace et commenter les différences observées.

(g) `mere(jackie, X), mere(X, ling).`
(h) `pere(abraham, X), mere(mona, X).`
(i) `pere(X, marge), mere(Y, marge).`
(j) `pere(X, marge), pere(X, Z).`
(k) `pere(X, Y).`

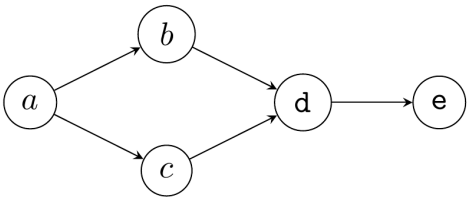
4. Quelle requête permet de trouver qui est la mère de la fille d'Abraham ?
5. Définir les prédicats suivants par des règles à ajouter dans le fichier puis les tester par des requêtes variées, en changeant les paramètres instanciés ou libres.

(a) `enfant(P, M, X)` qui est satisfait si `P` et `M` sont respectivement le père et la mère de `X`
(b) `grand_mere_paternelle(GM, X)`
(c) `parent(X, Y)` qui est satisfait si `X` est le père ou la mère de `Y`
(d) `grand_pere(GP, X)` qui est satisfait si `X` est un des grands-pères de `Y`
(e) `frere_ou_soeur(X, Y)`
(f) `tante(X, Y)`
(g) `cousin(X, Y)`
(h) `ancetre(X, Y)` qui est satisfait si `X` est un ancêtre de `Y` : l'un de ses parents, grands-parents, arrière-grands-parents, arrière-arrière-grand-parent, etc

Ainsi, on doit trouver 8 ancêtre à `bart` avec l'arbre généalogique fourni.

Exercice 18 – Graphe

On considère le graphe orienté sans cycle suivant :



1. En utilisant le prédicat binaire `arc`, représenter ce graphe par des faits en prolog.
2. Quelle requête permet d'afficher tous les nœuds accessibles depuis `a` ?
3. Quelle requête permet d'afficher tous les nœuds qui pointent vers `d` ?
4. Définir le prédicat `entre(X, S, D)` qui est satisfait si `X` est situé immédiatement entre `S` et `D`.

Par exemple, la requête `entre(X, a, d)` doit être satisfaite par `b` et `c`.
Par contre, les requêtes `entre(a, S, D)` `entre(X, a, b)` et `entre(X, a, e)` doivent échouer.

5. Définir le prédicat `chemin3(X, Y)` qui est satisfait si on peut aller de `X` à `Y` en au plus trois étapes.

Par exemple, la requête `chemin3(a, Y)` doit être satisfaite par `Y = b, c, d` ou `e`, la requête `chemin3(X, b)` doit être satisfaite uniquement par `X = a`.

6. Définir le prédicat `chemin(X, Y)` qui est satisfait si on peut aller de `X` à `Y`.
7. Que se passe-t-il si on ajoute un arc entre b et a ?

Thème 11 - TP

Objectifs

- prolog, listes, récursion

Exercices

Exercice 19 – Tests d'unification

Tester les unifications suivantes : calculer sur papier le résultat que prolog renvoie pour les requêtes suivantes, selon les principes d'unification et de construction de liste. Vérifier ensuite, en utilisant prolog, que vos réponses sont exactes, en examinant toutes les réponses successives lorsque plusieurs peuvent être obtenues.

1. `?- [a, b, c] = [X, Y].`
2. `?- [a, [b, c], d] = [X | Y].`
3. `?- [a, [b, c], d] = [X, [Y], Z].`
4. `?- [a, [b, c], d] = [X, [Y, Z], U].`
5. `?- [a, [b, c], d] = [X, [Y| Z], U].`
6. `?- [a, b, [c, d]] = [a, b | X].`
7. `?- [a, [b, c], d] = [a, b | X].`
8. `?- [a, b, c, d| X ] = [a, b | X].`
9. `?- append([a, X], [c, Y], [a, b, c, d]).`
10. `?- append([X], Y, [a, b, c, d]).`
11. `?- append(X, [Y], [a, b, c, d]).`
12. `?- append(X, Y, [a, b, c, d]).`

NB désormais, il est recommandé de nommer les variables qui doivent s'unifier avec des listes `L` ou `L1`, ... et de garder `X`, `Y`, ... pour des éléments des listes, afin de rendre les programmes plus lisibles.

Exercice 20 – Manipulation de listes

1. Implémenter le prédicat `premier` vu en TD et tester la requête `premier(L, a)`. Justifier le résultat obtenu.
2. Définir et tester de la même façon les prédicats `deuxieme` et `avantDernier`.
3. Définir le prédicat `inverse` tel que `inverse(L1, L2)` est satisfait si `L2` est la liste contenant les éléments de `L1` dans l'ordre inverse.

   Par exemple `inverse([a, b, c, d], L)` doit conduire à l'unique unification `L = [d, c, b, a]`
   et `inverse([], L)` à `L = []`.

Exercice 21 – Listes de courses et menus

Dans cet exercice, on considère que les constantes correspondent à des aliments et que les listes correspondent à des listes de courses ou à des menus, par exemple
`[orange, avocat, pomme, poulet, gateau, gateau, sardine, saumon, riz]` ou `[avocat, poulet, riz, yaourt]`.

On considère de plus que l'on dispose des prédicats unaires `entree`, `poisson`, `viande`, `accompagnement`, `fruit` et `dessert` qui permettent de définir des catégories d'aliments.

1. Télécharger le fichier `aliment.pl` depuis le moodle. Il contient un ensemble de faits pour les prédicats de catégories d'aliments.
2. Définir le prédicat `platPrincipal` tel que `platPrincipal(X)` est satisfait si `X` est une viande ou un poisson.
3. Ecrire une règle qui exprime que les fruits sont des desserts.
4. Définir le prédicat `menuComplet` tel que `menuComplet(L)` est satisfait si `L` est une liste qui décrit un menu complet, c'est-à-dire contenant, dans cet ordre, une entrée, un plat, un accompagnement et un dessert.
5. Quelle requête permet de trouver tous les menus complets possibles avec les aliments définis dans le fichier `aliment.pl` dont le plat principal est de la viande et le dessert un gâteau ?
6. Définir le prédicat `extraitEntreeMenuComplet` tel que `extraitEntreeMenuComplet(L, E)` est satisfait si `L` est une liste qui décrit un menu complet dont l'entrée est `E`.

   Par exemple, `extraitEntreeMenuComplet([avocat, poulet, riz, yaourt], E).` doit fournir pour unique unification `E = avocat`.
7. Définir de même le prédicat `extraitDessertMenuComplet` tel que `extraitDessertMenuComplet(L, D)` est satisfait si `L` est une liste qui décrit un menu complet dont le dessert est `D`.
8. Définir le prédicat `contient` tel que `contient(L, X)` est satisfait si une liste (de courses) `L` contient l'aliment `X` (permettant de vérifier qu'on n'a rien oublié au supermarché).

   Par exemple, pour
   `ListeCourses = [orange, avocat, pomme, poulet, gateau, gateau, sardine, saumon, riz]`,
   la requête `contient(ListeCourses, gateau).` doit réussir, alors que
   `contient(ListeCourses, yaourt).` doit échouer.
9. Définir le prédicat `extraitDessert` tel que `extraitDessert(L1, L2)` est satisfait si `L2` est la liste de tous les desserts présents dans la liste de courses `L1`.

   Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Ldessert)` doit conduire à l'unique unification `Ldessert = [orange, pomme, gateau, gateau].`
10. Définir le prédicat `supprimeDessert` tel que `supprimeDessert(L1, L2)` est satisfait si `L2` est la liste obtenue après suppression dans `L1` de tous les desserts qu'elle contient.

    Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Lres)` doit conduire à l'unique unification `Lres = [avocat, poulet, sardine, saumon, riz].`
11. Définir le prédicat `cureVitamineC` tel que `cureVitamineC(L1, L2)` est satisfait si `L2` est la liste obtenue en remplaçant dans `L1` tous les desserts par des oranges.

    Par exemple, pour la liste de courses de la question précédente, `extraitDessert(ListeCourses, Lres)` doit conduire à l'unique unification `Lres = [orange, avocat, orange, poulet, orange, orange, sardine, saumon, riz].`